INTRODUCTION TO NEURAL NETWORKS

Name: _____ Branch/Year: _____ Date: _____

Estimated Time: 90-120 minutes

WELCOME! Let's Build Your First "Brain Cell"

Imagine teaching a computer to make simple decisions, just like your brain does every day! We'll start with the SIMPLEST building block—a single artificial neuron called a "perceptron."

What You'll Learn: How to build a "smart decision maker" that learns from examples, just like you learn from experience!

Part 1: Understanding ONE Neuron (The Perceptron)

KEY IDEA: What is a Perceptron?

Think of it as a tiny decision-maker that:
1. Takes in some information (called "inputs")
2. Thinks about it (does some simple math)
3. Makes a YES or NO decision (called "output")

That's it! Really simple!

Task 1 — How Does a Neuron Work?

Step 1: Real Brain vs Computer Brain

Let's see how our brain cells (neurons) inspired artificial neurons!

TRY THIS: Match each part by filling in the blanks on the right!

Real Brain Part	What It Does	Computer Version
Dendrites	Receives signals from other neurons	**Inputs (x)**

Cell Body	Adds up all the signals	**Summation (z)**
Axon	Sends out the final decision	**Output (y)**
Synapse	Controls how strong each signal is	**Weights (w)**

Word Bank: Inputs, Weights, Summation (adding things), Output

Step 2: The Simple Math Behind It

> **KEY IDEA: The Perceptron Does Just 3 Simple Steps:**
>
> **Step A:** Multiply each input by its "weight" (importance)
> **Step B:** Add everything together (plus a "bias")
> **Step C:** Decide: If total ≥ 0, say YES (1). Otherwise, say NO (0)
>
> Don't worry! We'll practice this step-by-step!

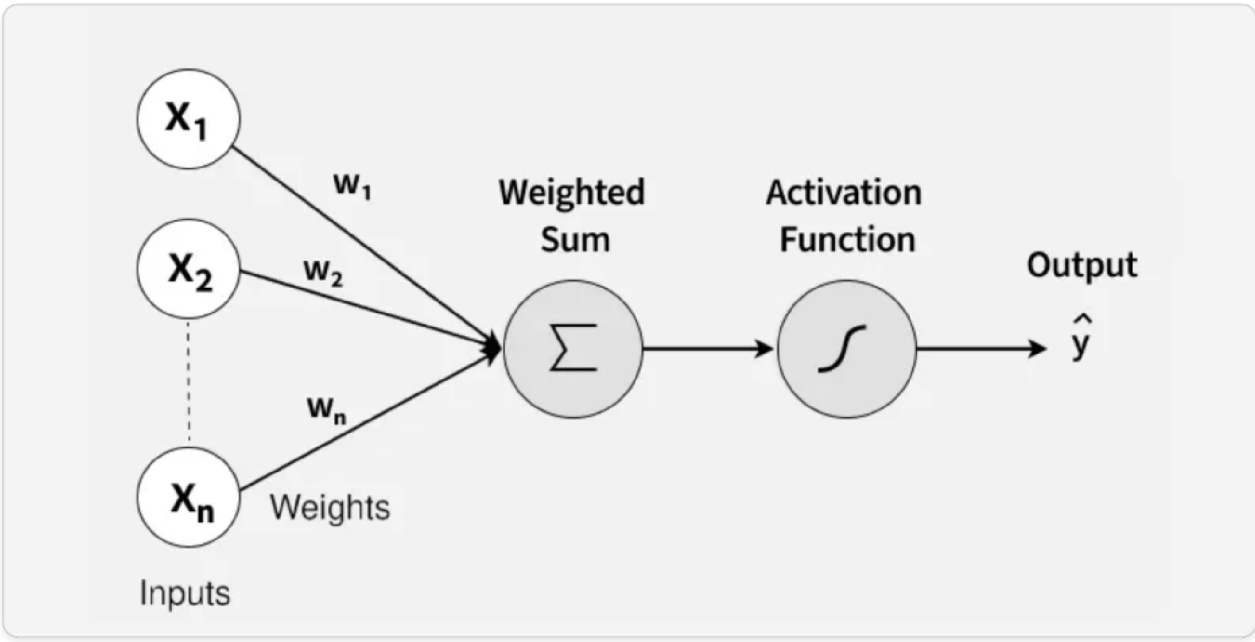

Figure 1: Visualizing the Perceptron Flow

The Formula (looks scary but it's easy!):

Step A & B: $z = (w_1 \times x_1) + (w_2 \times x_2) + b$

Step C: If $z \geq 0 \rightarrow$ Output = 1 (YES)
If $z < 0 \rightarrow$ Output=0 (NO)

> **EXAMPLE: Real-Life Example: Should I Order Pizza?**

Inputs (Information):
• x_1 = How hungry am I? (Scale: 0 to 10)
• x_2 = How much money do I have? (Scale: $0 to $50)

Weights (How important each factor is):
• w_1 = 2 (hunger is VERY important!)
• w_2 = 1 (money matters, but less than hunger)
• b = -15 (bias: I'm generally lazy about ordering)

Decision Rule:
If z ≥ 0 → ORDER PIZZA!
If z < 0 → Eat instant noodles instead

Step 3: Let's Practice Together! "Should I Go to Class?"

Imagine you built a perceptron to decide if you should go to class today:

The Inputs (Information):

- x_1 = Sleep: **8 hours** (You're well rested!)
- x_2 = Attendance: **90%** (You're a good student!)
- x_3 = Rain: **0** (It's sunny!)

The Weights (Importance):

- w_1 = 0.3 (Sleep makes you *slightly* want to skip to enjoy the day)
- w_2 = -0.5 (High attendance makes you want to GO, so it pushes towards "No Skip")
- w_3 = 0.8 (Rain makes you want to skip)
- b = 2.0 (Bias: Your inner laziness)

> **NOTICE: Wait, why a NEGATIVE weight?**
> Think of positive weights as "peer pressure to SKIP".
> Think of negative weights as your "conscience telling you to GO".
> Since high attendance is "good", it fights AGAINST skipping!

🧮 **CALCULATION TIME! (You do the math!)**

Step A & B: Calculate the "Vibe Score" (z)

z = (0.3 × 8) + (-0.5 × 90) + (0.8 × 0) + 2.0

z = (_____) + (_____) + (_____) + 2.0

z = (Calculate this!)

> **KEY IDEA: Step C: The Activation Function (The "Sigmoid Squisher")**
>
> **The Problem:** You probably got a HUGE negative number. Simply saying "No" isn't enough. We want a probability!

The Solution: The Sigmoid Function (σ)

$$\sigma(z) = 1 / (1 + e^{-z})$$

(where $e \approx 2.718$)

It acts like a **Dimmer Switch**. It "squishes" ANY number (no matter how big or small) into a range between **0 and 1**.

- Big Positive z → Output ≈ 1 (100% Yes)
- Big Negative z → Output ≈ 0 (0% Yes)
- Zero z → Output = 0.5 (50% Maybe)

Step D: The Final Calculation (Try it if you have a calculator!)

$\sigma(-40.6) = 1 / (1 + e^{40.6}) =$

Final Decision:

Your result is **0.000000000000000002...** (Not exactly zero, but very close!)

Verdict: There is an **almost 0% chance** you skip class. Go learn something! 🎓

Quick Check: Do You Understand So Far?

Q1: What does a perceptron do?

☐ A. Stores lots of data

☐ B. Makes YES/NO decisions based on inputs

☐ C. Displays pretty pictures

☐ D. Connects to WiFi

Q2: What are "weights" in a perceptron?

Answer: They control how each input is.

Q3: If z = 5, what does the perceptron output?

Answer: _____

(Remember: If z ≥ 0, output is 1)

Q4: True or False: A perceptron can learn from its mistakes.

Answer:

Part 2: Teaching the Perceptron to Learn (Gradient Descent)

THE SCENARIO: "The Marketing Guru 📈 "

You work for a startup. You have data on past ad campaigns, detailing how much was spent on TV vs. Social Media.

Goal: Teach the perceptron to look at ad spending and predict the **sales numbers**!

KEY IDEA: How Does Learning Actually Work? (Gradient Descent)

Imagine you are standing on top of a fog-covered mountain called **"Mount Error"**.
• **Height:** Represents your total mistake (Loss).
• **Goal:** Get to the bottom (Zero Error).
• **Problem:** You are blindfolded!

The Solution (Gradient Descent):
1. Feel the slope of the ground with your feet (Calculate Gradient).
2. Take a small step downhill (Update Weights).
3. Repeat until you reach the bottom! 📉

Task 2 — The Training Loop

The Data (Experience):

We have data from 5 past campaigns. We want to predict exact **Sales**.

#	TV Ad Cost ($k) [$x_1$]	Social Media Cost ($k) [$x_2$]	Sales Units Sold (k) [y]
1	230.1	37.8	22.1
2	44.5	39.3	10.4
3	17.2	45.9	9.3
4	151.5	41.3	18.5
5	180.8	10.8	12.9

> **NOTICE: Goal:** Teack the Neural Network to look at ad spending and **PREDICT** the Sales number. This is called **REGRESSION** (predicting a number) instead of Classification (Yes/No).

The "Step Downhill" Formula (Gradient Descent)

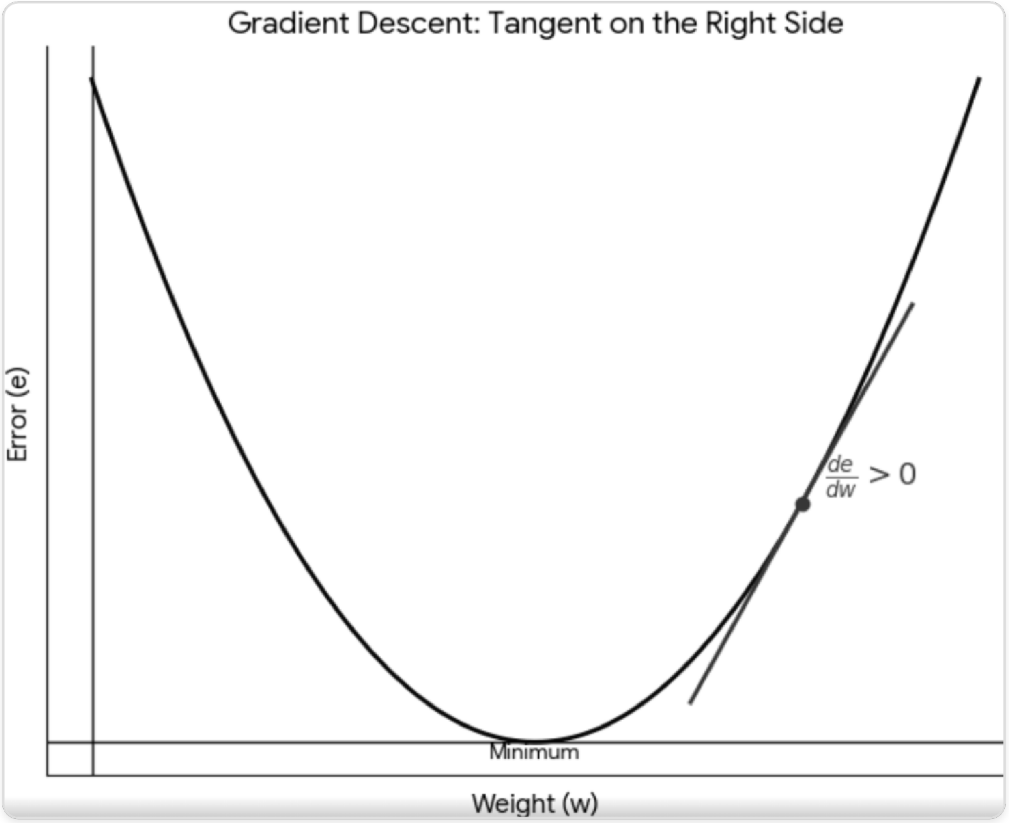

Figure 2: Finding the Minimum Error by checking the Slope

> **KEY IDEA: The Mathematical Logic (The "Blame Game" 🕵)**
>
> We measure our total mistake as **Squared Error (e)**:
> **$e = (y - y_p)^2$**
>
> Expanding the prediction (y_p):
> **$e = (y - (w_0 + w_1x_1 + w_2x_2))^2$**
>
> **Who is Responsible? (Differentiation - de/dw)**
> We check the "Slope" (Derivative) for each weight to see how much it affects the error:
>
> • Change due to Bias (w_0): **$de/dw_0 = -2(y - y_p)$**
> • Change due to TV Weight (w_1): **$de/dw_1 = -2(y - y_p) * x_1$**
> • Change due to Social Weight (w_2): **$de/dw_2 = -2(y - y_p) * x_2$**
>
> **The Update Rule (Stepping Down):**
> To reduce error, we move *opposite* to the slope (Gradient Descent):
> **w_0_new = w_0_old - (Learning Rate * de/dw_0)**

> *(This automatically corrects the weight regardless of whether it's too high or too low!)*

Let's Apply This Logic:

Let's Train! (One Step of Gradient Descent)

Scenario: We are training on **Campaign #2** (Low TV, High Social).

- x_1 (TV) = **44.5**
- x_2 (Social) = **39.3**
- Actual Sales (y) = **10.4**

Current Brain State (Random Weights):

- w_1 (TV weight) = **0.1**
- w_2 (Social weight) = **0.2**
- bias = **0**

Step 1: Make a Prediction (Forward Pass)

Prediction = $(w_1 \cdot x_1) + (w_2 \cdot x_2) + b$

Prediction = $(0.1 \cdot 44.5) + (0.2 \cdot 39.3) + 0$

Prediction = 4.45 + 7.86 = **12.31**

> **NOTICE:** The AI predicted **12.31k** sales.

Step 2: Calculate Error

Error = Actual - Predicted

Error = 10.4 - 12.31 = **-1.91**

(The AI guessed too high! It needs to lower its weights.)

Step 3: Update Weights (The Learning)

We use a small **Learning Rate (0.0001)**. *Why? Because if we take huge steps, we might overshoot the bottom of the mountain!*

Common Term (Gradient part): $-2(y - y_p) = -2(10.4 - 12.31) = -2(-1.91) =$ **3.82**

Update w_1 (TV):

$slope_1 = 3.82 \times 44.5 = 170.1$

w_1_new = 0.1 - (**0.0001** × 170.1) = 0.1 - 0.017 = **0.083**

(We subtract: Learning Rate × Slope)

Update w_2 (Social):

$slope_2 = 3.82 \times 39.3 = 150.1$

w_2_new = 0.2 - (**0.0001** × 150.1) = 0.2 - 0.015 = **0.185**

> **KEY IDEA: Result:** Both weights decreased slightly. The Learning Rate ensured we took a **safe, small step** towards the correct answer.

Step 4: Did It Learn? (Check New Prediction)

Let's try again with the NEW weights:

New Pred = (0.083 × 44.5) + (0.185 × 39.3)

New Pred = 3.69 + 7.27 = **10.96**

EXAMPLE: IMPROVEMENT REPORT:

- **Actual Sales:** 10.4
- **Old Prediction:** 12.31 (Error: -1.91)
- **New Prediction:** 10.96 (Error: -0.56)

SUCCESS! The error got smaller (closer to zero)!
The Neural Network is strictly better than it was a second ago.

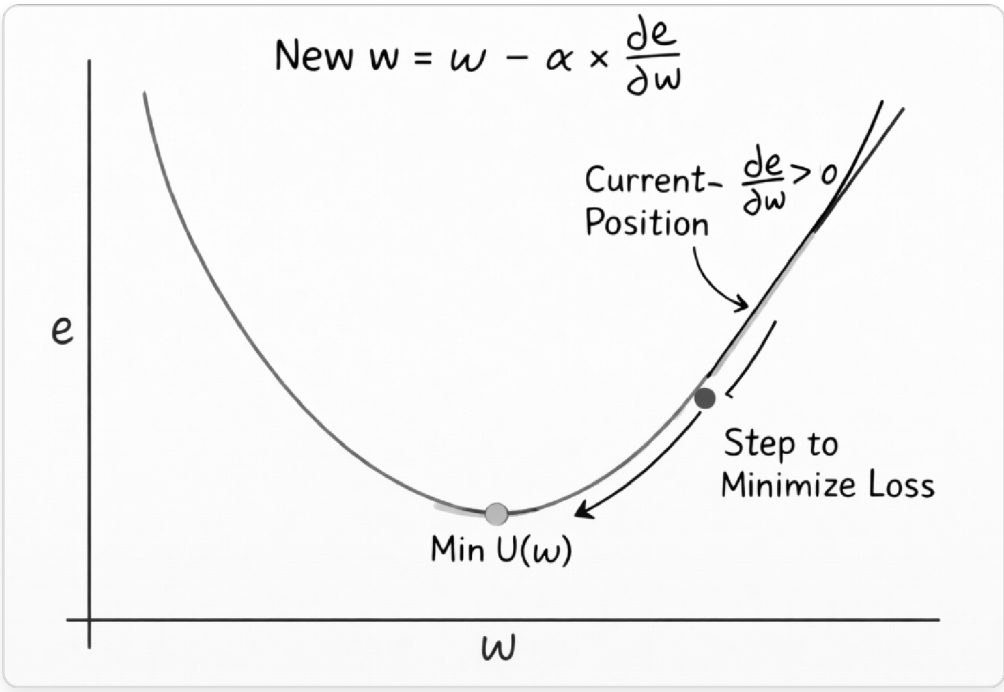

Figure 3: We successfully took a step down the mountain!

What just happened? (See Figure 3)

- We were at the **"Current Position"** (High Error).
- We calculated the slope (**de/dw**).
- We took a step in the opposite direction.

> - We landed closer to the **"Min U(w)"** (Minimum Error)!

Part 3: Multi-Layer Perceptron (The "Deep" Brain)

THE PROBLEM: "Simple Lines Aren't Enough"

A single perceptron is great, but it's simple-minded. It draws **ONE straight line** to make decisions.

Real life is curvy and complex! We need teamwork.

Task 3 — Meet the Neural Network

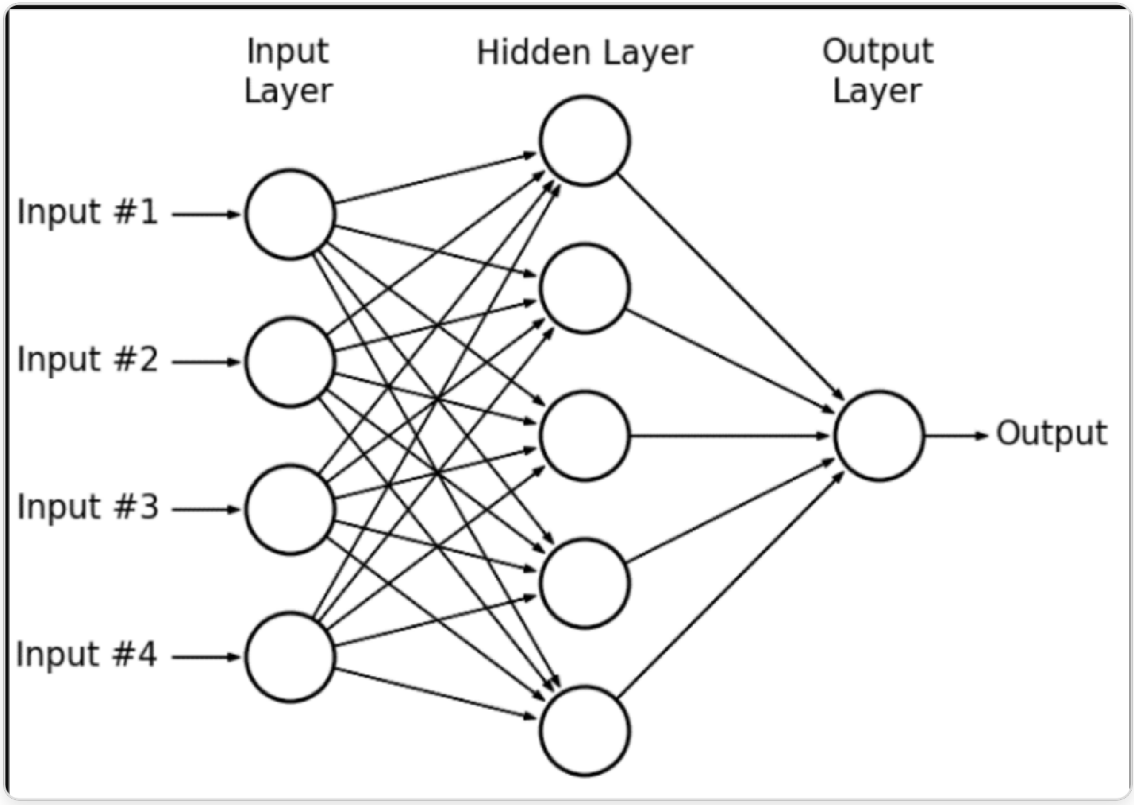

Figure 4: A Network of Perceptrons working together

> **KEY IDEA: The "Pizza Factory" Analogy (Hidden Layers)** 🍕
>
> A single ingredient (Flour) isn't a pizza. You need steps!
>
> 1. **Input Layer (Raw Ingredients):** Flour, Water, Tomato, Spices.
> 2. **Hidden Layer (The Kitchen Chefs):**
> - Chef A mixes Flour + Water → Creates **Dough Base**.
> - Chef B mixes Tomato + Spices → Creates **Secret Sauce**.

> > *(They create "features" that are useful!)*
> > 3. **Output Layer (The Server):** Combines Dough + Sauce + Cheese → **FINAL PIZZA!**
> >
> > **Why "Hidden"?** When you tackle the pizza at your table (Output), you don't see the chefs working in the kitchen (Hidden Layer). You only see the menu and the meal!

Congratulations!

> **KEY IDEA: What You've Learned:**
>
> - How a single perceptron makes decisions (like a brain cell!)
> - How to calculate Weighted Sums and use the Sigmoid function.
> - How Gradient Descent minimizes error (finding the valley!).
> - Why we need **Hidden Layers** to solve complex problems.
>
> **You now understand the core mechanics of AI!**